Contracts Erasure Theorem

Define erasure compiler E as a homomorphic function over expressions of Contracts with a single transformation:

$$\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau \rightarrowtail e$$

Relation

Define Contracts F as contexts without $\mathsf{check}_j^k(E, v)$.

Define these relations:

- $R_e \subseteq$ Contracts $e \times$ Base e

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $F \times$ Base E

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

- $R_v \subseteq$ Contracts $v \times$ Base v

  If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We define $\tilde{v}$ as shorthand for any variable $\tilde{v}$ satisfying $v \sim \tilde{v}$.

We define context F that is contexts without $\mathsf{check}_j^k(E, v)$.

$\boxed{\sim \subseteq \text{Contracts } F \times \text{Base } E}$

Define $\tilde{F}$ such that $F \sim \tilde{F}$.

$$\begin{aligned} \square &\sim \square \\ F\ e &\sim \tilde{F}\ \tilde{e} \\ v\ F &\sim \tilde{v}\ \tilde{F} \\ \mathsf{mon}_j^{k,l}(\kappa, F) &\sim \tilde{F} \end{aligned}$$

$\boxed{\sim \subseteq \text{Contracts } e \times \text{Base } e}$

$$\begin{aligned} v &\sim \tilde{v} \\ e_1\ e_2 &\sim \tilde{e}_1\ \tilde{e}_2 \\ \mathsf{mon}_j^{k,l}(\kappa, e_2) &\sim \tilde{e}_2 \\ \mathsf{check}_j^k(e_1, v) &\sim \tilde{v} \end{aligned}$$

$\sim \subseteq$ Contracts $v \times$ Base v

$$
\begin{array}{rcl}
n & \sim & n \\
b & \sim & b \\
x & \sim & x \\
\lambda x.e & \sim & \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa, v) & \sim & \tilde{v}
\end{array}
$$

Lemma 0.1. (Relation is closed under check-free contexts)

If $e \sim \tilde{e}$ and $F \sim \tilde{F}$, then $F[e] \sim \tilde{F}[\tilde{e}]$.

Proof.

By induction on $F \sim \tilde{F}$.

- Case $\square \sim \square$

$e \sim \tilde{e}$	Given
$F[e] = e$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{e}$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	

- Case $F_1\ e_2 \sim \tilde{F}_1\ e_2$

$e \sim \tilde{e}$	Given
$F_1[e] \sim \tilde{F}_1[\tilde{e}]$	I.H.
$F[e] = F_1[e]\ e_2$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{F}_1[\tilde{e}]\ \tilde{e}_2$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	definition

- Similarly for other cases

□

Lemma 0.2. (Relation Decomposition Through Contexts)

If $e_1 \sim e_1'$, $e_1 = E[e_2]$, and e_2 is a redex, then there are $\tilde{E}$ and $\tilde{e}_2$ such that $E \sim \tilde{E}$ and $e_2 \sim \tilde{e}_2$ and $e_1' = \tilde{E}[\tilde{e}_2]$.

Proof. By induction on the check-free context E.

- Case $E = \square$

$e_1 = e_2$	Context Plug in

$\tilde{E} = \square$	Choose
$\tilde{e}_2 = e'_1$	Choose
$E \sim \tilde{E}$	Definition
$e_2 \sim \tilde{e}_2$	Given
$e'_1 = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = E_0\ e_0$

$e_1 = E_0[e_2]\ e_0 \sim e'_1$	Given
$E_0[e_2] \sim e'_0$	Relation inversion
$e_0 \sim \tilde{e}_0$	Relation inversion
$e'_1 = e'_0\ \tilde{e}_0$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e'_0 = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{E}_0\ \tilde{e}_0$	Choose
$E \sim \tilde{E}$	Definition
$e'_1 = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = v\ E_0$

$e_1 = v\ E_0[e_2] \sim e'_1$	Given
$v \sim \tilde{v}$	Relation inversion
$E_0[e_2] \sim e'_0$	Relation inversion
$e'_1 = \tilde{v}\ e'_0$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e'_0 = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{v}\ \tilde{E}_0$	Choose
$E \sim \tilde{E}$	Definition
$e'_1 = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = \mathsf{mon}_j^{k,l}(\kappa, E_0)$

$e_1 = \mathsf{mon}_j^{k,l}(\kappa, E_0[e_2]) \sim e'_1$	Given
$E_0[e_2] \sim e'_1$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e'_1 = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{E}_0$	Choose
$E \sim \tilde{E}$	Definition

$e_1' = \tilde{E}[\tilde{e}_2]$ Context Plug in

- Other check-free context cases follow similarly.

□

Lemma 0.3. (Relation is closed under substitution)

If $e \sim \tilde{e}$ and $v \sim \tilde{v}$, then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$ for every x.

Proof.

By I.H. on $e \sim \tilde{e}$.

- Case $x \sim x$

$v \sim \tilde{v}$ Given
$e[x := v] = v$ Replacement
$\tilde{e}[x := \tilde{v}] = \tilde{v}$ Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$ By $v \sim \tilde{v}$

- Case $y \sim y$, where $y \neq x$

$v \sim \tilde{v}$ Given
$e[x := v] = y$ Replacement
$\tilde{e}[x := \tilde{v}] = y$ Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$ By $y \sim y$

- Case $e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$

$e_1[x := v] \sim \tilde{e}_1[x := \tilde{v}]$ I.H.
$e_2[x := v] \sim \tilde{e}_2[x := \tilde{v}]$ I.H.
$e[x := v] = e_1[x := v]\ e_2[x := v]$ Replacement
$\tilde{e}[x := v] = \tilde{e}_1[x := \tilde{v}]\ \tilde{e}_2[x := \tilde{v}]$ Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$ Definition

- Similarly with other cases. Lambda case avoid same variable by replacement $e[x := v]$ to free variables of e.

□

Lemma 0.4. (Evaluation-context decomposition)

Forall context E, either:

- $E = F$ or
- $E = F\left[\mathsf{check}_j^k(E', v)\right]$

Proof. By induction on E.

- Case $\square$

$F = \square$ Choose
$E = F$

- Case $E = \mathsf{check}_j^k(E_1, v)$

  $F = \square$ Choose
  $E' = E_1$ Choose
  $E = F\left[\mathsf{check}_j^k(E_1, v)\right]$

- Case $E_1\ e$

  I.H. on E_1:

  - Case $E_1 = F_1$

    $E = F_1\ e$
    $F = F_1\ e$ Choose
    $E = F$

  - Case $E_1 = F_1\left[\mathsf{check}_j^k(E_1', v)\right]$

    $E = F_1\left[\mathsf{check}_j^k(E_1', v)\right]\ v$
    $F = F_1\ v$ Choose
    $E' = E_1'$ Choose
    $E = F\left[\mathsf{check}_j^k(E_1', v)\right]$

- Other cases follows similarly

□

We define the predicate that will be used for simulation. The beauty of the predicate $\mathrm{ER}(e, e')$ is that $e = E\left[\mathsf{blame}_j^k\right]$ returns false, so we can skip this case in simulation proof.

$$\mathrm{ER}(e, e') = \begin{cases} \text{true if } e \sim e' \text{ and } e \neq F\left[\mathsf{check}_j^k(e_0, v)\right] \\ \text{true if } e = F\left[\mathsf{check}_j^k(e_0, v)\right] \text{ and } e' = \tilde{F}[\tilde{v}] \\ \text{false otherwise} \end{cases}$$

Lemma 0.5. (Erasure satisfies the state predicate)

If $e' = \mathrm{erase}(e)$, then $\mathrm{ER}(e, e')$.

Proof. I.H on $e' = \mathrm{erase}(e)$.

Because the compiler doesn't compile check (runtime expression), we never get the second clause of EQ. Therefore, $e \neq F\left[\mathsf{check}_j^k(e_0, v)\right]$, and all child nodes of e also not this form.

- Case $e' = \mathrm{erase}\left(\mathsf{mon}_j^{k,l}(\kappa, e_1)\right) = \mathrm{erase}(e_1) = e_1'$

  $\mathrm{ER}(e_1, e_1')$ I.H.

$e_1 \sim e_1'$	First clause EQ
$\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim \tilde{e}_1$	Definition
$e_1 \sim \tilde{e}_1$	Definition
$\tilde{e}_1 = e_1'$	Choose
$\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim e_1'$	
$\mathrm{ER}(e, e')$	First clause EQ

- Case $e' = \mathrm{erase}(e_1\ e_2) = \mathrm{erase}(e_1)\ \mathrm{erase}(e_2) = e_1'\ e_2'$

$\mathrm{ER}(e_1, e_1')$	I.H.
$\mathrm{ER}(e_2, e_2')$	I.H.
$e_1 \sim e_1'$	First clause EQ
$e_2 \sim e_2'$	First clause EQ
$e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$	Definition
$\mathrm{ER}(e, e')$	First clause EQ

- Other cases follow similarly to the above case.

□

Main Theorem Proof

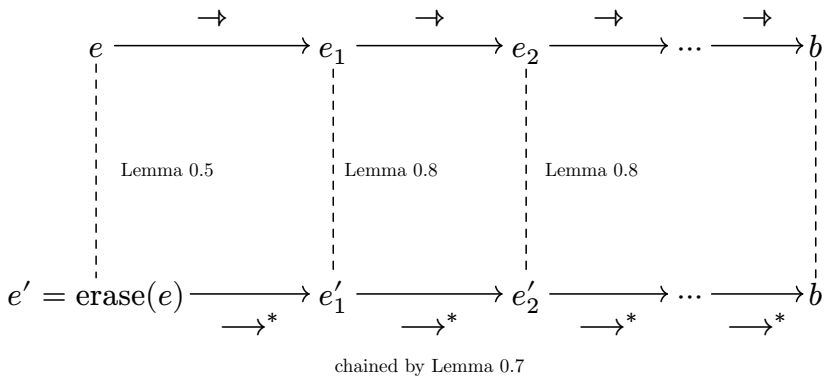


Figure 1: Structure of the contract erasure proof. Dashed vertical lines denote ER.

Theorem 0.6. (Contract Erasure Theorem)

If $e' = \mathrm{erase}(e)$ and $e \twoheadrightarrow^* b$, then $e' \longrightarrow^* b$.

Proof.

$\mathrm{ER}(e, e')$	Lemma 0.5
$e' \longrightarrow^* b$	Lemma 0.7

□

Lemma 0.7. (Erasure Multistep Simulation)

If $\mathrm{ER}(e, e')$ and $e \twoheadrightarrow^* b$ then $e' \longrightarrow^* b$.

$\{(e, e') \mid e \twoheadrightarrow^* b \text{ and } (e, e') \in \text{EQ}\}$. This active filter prevents $e \twoheadrightarrow^* \mathsf{blame}_j^k$ as well as anything not b. Mainly we worry about $E\left[\mathsf{blame}_j^k\right] \to \mathsf{blame}_j^k$ and $E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right] \to E\left[\mathsf{blame}_j^k\right]$. It's a strong claim here that $e \twoheadrightarrow^* b$ so we can ignore these states. But the single step simulation must be reworded correctly.

Now if we check the argument for EQ, we can actually filter out $E\left[\mathsf{blame}_j^k\right]$ because it doesn't fit two clauses. So we only care about the check expression.

Proof.

By induction on $e \twoheadrightarrow^* b$.

- Case $b \twoheadrightarrow^* b$

$\text{ER}(e, e')$	Given
$e \sim e'$	First clause EQ
$e' = b$	Relation inversion $b \sim e'$
$e' \longrightarrow^* b$	Reflexivity

- Case $e \to e_1$ and $e_1 \twoheadrightarrow^* b$

  By cases on $e \to e_1$.

  - Case $e = E\left[\mathsf{blame}_j^k\right] \to \mathsf{blame}_j^k = e_1$ for some $E \neq \square, k, j$

$e_1 \twoheadrightarrow^* \mathsf{blame}_j^k$	Reflexivity
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

  - Case $e = E[e_2] \to E[e_3] = e_1$ where $e_2 \dashrightarrow e_3$ is a redex, and $e_2 \neq \mathsf{blame}_j^k$.

$\text{ER}(e, e')$	Given
$e \to e_1$	Given
$e \neq E\left[\mathsf{blame}_j^k\right]$	Case assumption

    – Case $e_2 = \mathsf{check}_j^k(\mathsf{false}, v)$, and $e_3 = \mathsf{blame}_j^k$

$e_1 \to \mathsf{blame}_j^k$	Step-C
$e_1 \twoheadrightarrow^* \mathsf{blame}_j^k$	
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

    – Other cases, $e_2 \neq \mathsf{check}_j^k(\mathsf{false}, v)$, and $e_3 \neq \mathsf{blame}_j^k$

    Relation is not local so we run simulation on $e \to e_1$.

$e_1 \neq E\left[\mathsf{blame}_j^k\right]$
$e' \longrightarrow^* e_1'$ Lemma 0.8
$\mathrm{ER}(e_1, e_1')$ Lemma 0.8
$e_1' \longrightarrow^* b$ I.H.
$e' \longrightarrow^* b$ Transitive

□

Lemma 0.8. (Erasure Step Simulation)

If $\mathrm{ER}(e_1, e_1')$, $e_1 \twoheadrightarrow e_2$, and $e_2 \neq E_2\left[\mathsf{blame}_j^k\right]$, then there is e_2' such that $e_1' \longrightarrow^* e_2'$ and $\mathrm{ER}(e_2, e_2')$.

Proof.

By case analysis of $\mathrm{ER}(e_1, e_1')$:

- Case $e_1 = F\left[\mathsf{check}_{j_1}^{k_1}(e_{1a}, v)\right]$ and $e_1' = \tilde{F}[\tilde{v}]$
  - Case $e_{1a} = \mathsf{true}$

    $e_1 \twoheadrightarrow F[v]$ Step-C
    $e_2 = F[v]$
    $e_2 \sim \tilde{F}[\tilde{v}]$ Definition
    $e_1' \longrightarrow^* \tilde{F}[\tilde{v}]$ Reflexivity
    $e_2' = \tilde{F}[\tilde{v}]$ Choose
  - Case $e_{1a} = \mathsf{false}$
  - Case $e_{1a} \dashrightarrow e_{1b}$

    $e_1 \twoheadrightarrow F\left[\mathsf{check}_{j_1}^{k_1}(e_{1b}, v)\right]$ Step-C
    $e_2 = F\left[\mathsf{check}_{j_1}^{k_1}(e_{1b}, v)\right]$
    $e_2 \sim \tilde{F}[\tilde{v}]$ Definition
    $e_1' \longrightarrow^* \tilde{F}[\tilde{v}]$ Reflexivity
    $e_2' = \tilde{F}[\tilde{v}]$ Choose
- Case $e_1 \sim e_1'$ and $e_1 \neq F\left[\mathsf{check}_{j_1}^{k_1}(e_{1a}, v)\right]$

  Casing on $e_1 \twoheadrightarrow e_2$:
  - Case $E\left[\mathsf{blame}_{j_2}^{k_2}\right] \twoheadrightarrow \mathsf{blame}_{j_2}^{k_2}$ contradiction.
  - Case $E[e_3] \twoheadrightarrow E[e_4]$ where $e_3 \dashrightarrow e_4$ is a redex.

    By Lemma 0.4,
    - Case $E = F\left[\mathsf{check}_{j_3}^{k_3}(E_3, v_3)\right]$

$e_1 = F\left[\mathsf{check}^{k_3}_{j_3}(E_3[e_3], v_3)\right]$ contradiction.

– Case $E = F$ with F a non-check context. Without confusion, we use E below:

For all cases, we have $E \sim \tilde{E}, e_3 \sim \tilde{e}_3, e_4 \sim \tilde{e}_4$ by Lemma 0.2.

By cases on the redex reduction $e_3 \dashrightarrow e_4$:

- Case $(\lambda x.e_5)\ v_5 \dashrightarrow e_5[x := v_5]$

$e_3 \sim (\lambda x.\tilde{e}_5)\ \tilde{v}_5$	Definition
$\tilde{e}_3 = (\lambda x.\tilde{e}_5)\ \tilde{v}_5$	$e_3 \sim \tilde{e}_3$
$(\lambda x.\tilde{e}_5)\ \tilde{v}_5 \dashrightarrow \tilde{e}_5[x := \tilde{v}_5]$	`S-App`
$e_4 \sim \tilde{e}_5[x := \tilde{v}_5]$	Lemma 0.3
$\tilde{e}_4 = \tilde{e}_5[x := \tilde{v}_5]$	Lemma 0.3
$E[e_3] \sim \tilde{E}[\tilde{e}_3]$	Lemma 0.1
$e'_1 = \tilde{E}[\tilde{e}_3] \longrightarrow^* \tilde{E}[\tilde{e}_4] = e'_2$	Step and Closure
$e_2 = E[e_4] \sim \tilde{E}[\tilde{e}_4] = e'_2$	Lemma 0.1

- Case $\mu x : \tau.e_3 \dashrightarrow e_3[x := \mu x : \tau.e_3]$

Similar to the preceding case.

For below cases, we left out obvious rules about relation.

- Case $\mathsf{check}^k_j(\mathsf{true}, v) \dashrightarrow v$

$\tilde{e}_3 = \tilde{v}$	Definition
$e_4 \sim \tilde{v}$	Definition
$e'_1 \longrightarrow^* \tilde{E}[\tilde{v}] = e'_2$	Reflexivity
$e_2 \sim e'_2$	Lemma 0.1

- Case $\mathsf{check}^k_j(\mathsf{false}, v) \dashrightarrow \mathsf{blame}^k_j$

$e_2 = E\left[\mathsf{blame}^k_j\right]$	`C-Step`
$e_2 \neq E_2\left[\mathsf{blame}^{k_2}_{j_2}\right]$	Given
$\bot$	Contradiction

- Case $\mathsf{mon}^{k,l}_j(\mathsf{flat}(e_3), v_3) \dashrightarrow \mathsf{check}^k_j(e_3\ v_3, v_3)$

$\tilde{e}_3 = \tilde{v}_3$	Definition
$e_4 \sim \tilde{v}_3$	Definition
$e'_1 \longrightarrow^* \tilde{E}[\tilde{v}_3] = e'_2$	Reflexivity
$e_2 \sim e'_2$	Lemma 0.1

- Case $\mathsf{mon}^{k,l}_j(\kappa, v_3) \dashrightarrow \mathsf{guard}^{k,l}_j(\kappa, v_3)$

$\tilde{e}_3 = \tilde{v}_3$	Definition

$e_4 \sim \tilde{v}_3$ Definition
$e_1' \longrightarrow^* \tilde{E}[\tilde{v}_3] = e_2'$ Reflexivity
$e_2 \sim e_2'$ Lemma 0.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

  $\tilde{e}_3 = \tilde{v}_1\ \tilde{v}_2$ Definition
  $\tilde{e}_4 \sim \tilde{v}_1\ \tilde{v}_2$ Definition
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_1\ \tilde{v}_2] = e_2'$ Reflexivity
  $e_2 \sim e_2'$ Lemma 0.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$, where $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

  Similar to the preceding case.

- Other cases (Base) follows similarly.

□

Correction note for Lemma 0.8. The following paragraphs replace the indicated reasoning when completing the proof; the original proof above is deliberately left unchanged.

First ER clause: obtain the related context before analyzing redexes. In the case $e_1 \sim e_1'$ and $e_1 \neq F\left[\mathsf{check}_j^k(e_0, v)\right]$, invert the source step. Step-C-Blame is impossible because no blaming expression satisfies `ereq`. Hence $e_1 = E[e_3]$, $e_2 = E[e_4]$, and $e_3 \dashrightarrow e_4$. Apply Lemma 0.4 to E. If $E = F\left[\mathsf{check}_j^k(E_0, v_0)\right]$, then $e_1 = F\left[\mathsf{check}_j^k(E_0[e_3], v_0)\right]$, contradicting the first ER clause. Therefore E is check-free. Moreover e_3 cannot itself be a check redex, for otherwise $e_1 = E\left[\mathsf{check}_j^k(e_0, v)\right]$, the same contradiction.

Now apply Lemma 0.2 to $e_1 \sim e_1'$ and $e_1 = E[e_3]$. It supplies witnesses $\tilde{E}$ and $\tilde{e}_3$ such that $E \sim \tilde{E}$, $e_3 \sim \tilde{e}_3$, and $e_1' = \tilde{E}[\tilde{e}_3]$. It does **not** supply a relation for e_4; that relation is proved separately in each redex case.

For example, for $e_3 = (\lambda x.e_5)\ v_5$, inversion of $e_3 \sim \tilde{e}_3$ yields $\tilde{e}_3 = (\lambda x.\tilde{e}_5)\ \tilde{v}_5$, with $e_5 \sim \tilde{e}_5$ and $v_5 \sim \tilde{v}_5$. S-App gives the target reduct $\tilde{e}_5[x := \tilde{v}_5]$, while Lemma 0.3

gives $e_4 = e_5[x := v_5] \sim \tilde{e}_5[x := \tilde{v}_5]$. Closing this relation under $E \sim \tilde{E}$ proves the ER conclusion. The μ and other base-language redexes are similar.

Check redexes belong only to the second ER clause. In the first clause, check redexes have already been ruled out. In the second clause, $e_1 = F\left[\mathsf{check}^k_j(e_{1a}, v)\right]$ and $e'_1 = \tilde{F}[\tilde{v}]$:

- If $e_{1a} = \mathsf{true}$, choose $e'_2 = e'_1$. Then $e_2 = F[v] \sim \tilde{F}[\tilde{v}]$ by Lemma 0.1, so ER holds by its first clause.
- If $e_{1a} \dashrightarrow e_{1b}$, choose $e'_2 = e'_1$. Then $e_2 = F\left[\mathsf{check}^k_j(e_{1b}, v)\right]$, so ER holds by its second clause.
- If $e_{1a} = \mathsf{false}$, then $e_2 = F\left[\mathsf{blame}^k_j\right]$, contradicting the premise that e_2 is not a blaming expression.